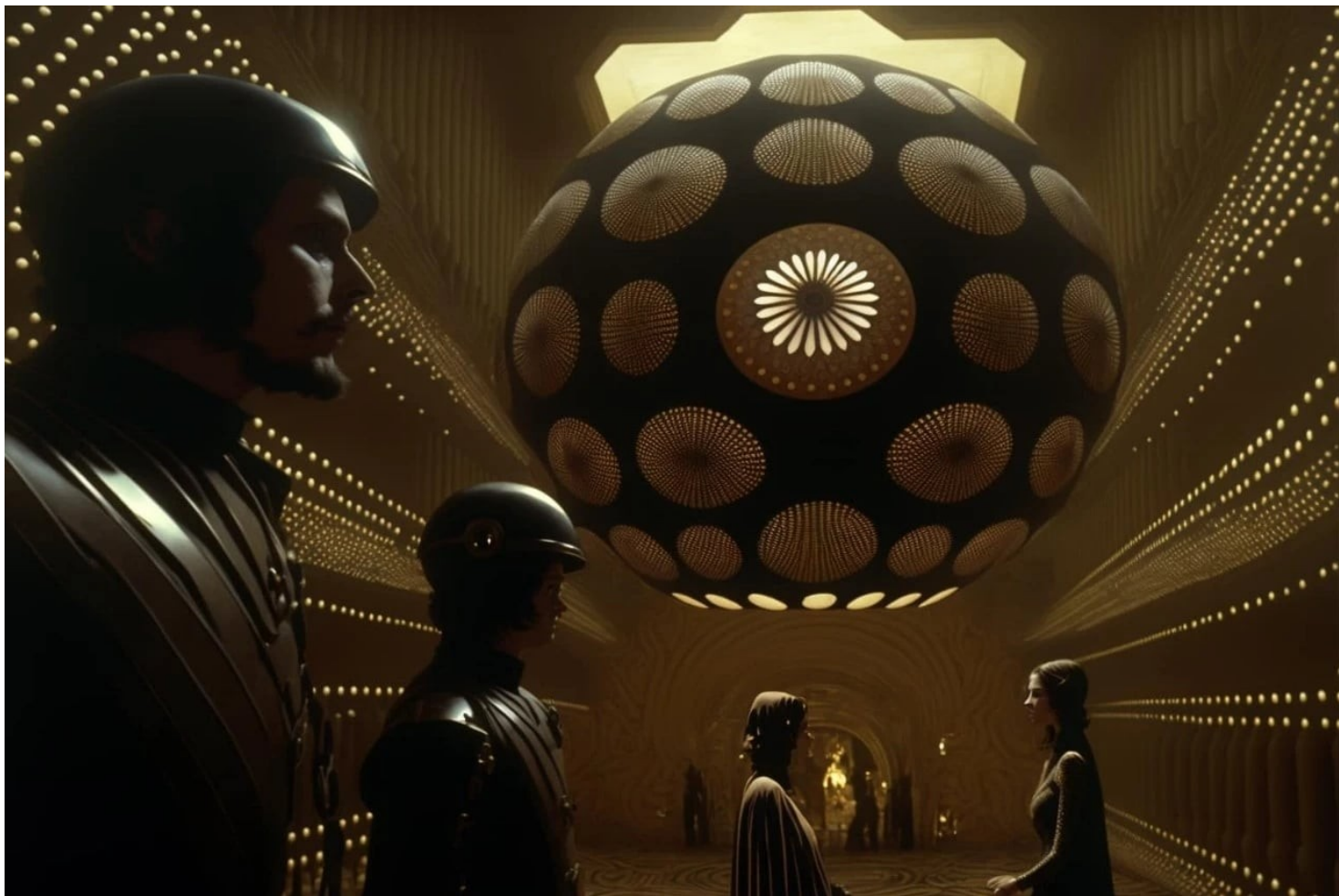




mentat

A library to define a graph of computation with
abstract serverless entities.

Spec

```
name: example

events:
  ProcessMessage:
    channel: my-channel

routes:
  /api/foo:
    function: f1

functions:
  f1:
    runtime:
      lang: python3.14
    code: |
      def handler(event, context):
        return event
  f2:
    runtime:
      lang: python3.14
      uri: "git://github.com/myorg/f2.git"
  f3:
    runtime:
      lang: python3.14
      uri: ./f3

channels:
  my-channel:
    handler: default
```

→ Inlined

→ Vended

→ Filesystem

Library

```
let spec = TopologySpec {
  name: "foo",
  functions: vec![
    FunctionSpec {
      name: "f1",
      uri: "../f1",
      runtime: RuntimeSpec {
        handler: "handler.handler",
        lang: Lang::Python311
      },
      build: BuildSpec {
        kind: BuildKind::Inline
      }
    }
  ],
  events: vec![
    EventSpec {
      name: "e1",
      function: "f1"
    }
  ]
};

let opts = ComposeOpts { };
let topology = compose(&spec, opts);
topology.validate()
```

- Mentat is a Rust library with PyO3 bindings and a Lisp interpreter.
- **compose** generates a proper graph that is optimized for breadth-first search (BFS). No filesystem walking needed.
- **compose** generates a graph where nodes are entities and edges are provider-specific templated annotations.
- Differ is part of mentat where the diffs are structural and not filesystem-based
- Making it a library allows agents to dynamically generate topologies inside lambda/agentcore functions

Lisp

Lisp allows us to express topologies and the composition of entities in a richer way.

Lisp is optional and for advanced use. YAML is for common use.

```
(define-event e1)

(define-event e2)

(define-function f1
  :uri "../fns/1"
  :runtime {:lang "python3.11"}
  :build {:kind 'inline})

(define-function f2
  :uri "git://myrepo/f2.git"
  :runtime {:lang "ruby3.3"}
  :build {:kind 'inline})

(define-function f3
  :uri "git://myrepo/f3.git"
  :runtime {:lang "clojure11.2"})

(define-route r1
  :path "/api/test"
  :async true)

(compose e1 f1 e2)

(compose r1 f1 f2 f3)
```

```
cat topology.lisp | ./mt
```

```
./mt --repl
```

```
> (first functions)
```

```
> (map :uri functions)
```

Output - Documentation

```
mt --provider aws --format tree  
mt --provider aws --format icepanel  
mt --provider aws --format json  
mt --provider aws --format bincode > mytopology.mt
```

Generate architecture descriptions using C4/icepanel or other standard forms.

The composed topology is the architecture description.

Output - IaC

```
mt --provider aws --format tf
mt --provider aws --format cdk
mt --provider aws --format rust
mt --provider aws --format rust-tc | tc create -s sandbox -e env --
mt --provider aws --format python
```

We can generate Skills and prompts dynamically for the composed topology and have LLMs (or devops-agents) generate the tc-equivalent in python, cdk, tf!

The generated infra code (IaC) is the manifest and is sandboxed, versioned etc.

Besides generating IaC, it can be used for sophisticated workflows – testing, documentation etc.